

Algorithm

Step-by-Step Procedure :

1. Collect MRI brain scan images from the dataset, organized into four class folders: glioma, meningioma, notumor, and pituitary.
2. Load the Training and Testing image folders separately using their directory paths.
3. Read each image using OpenCV and check its width, height, and number of color channels.
4. Resize all images to a fixed, uniform size so they can be fed into the neural network consistently.
5. Normalize pixel values (scale them between 0 and 1) to help the model train faster and more accurately.
6. Assign a label to each image based on its folder name (glioma, meningioma, notumor, or pituitary).
7. Split the data into Training and Testing sets (already provided as separate folders in this project).
8. Build a Convolutional Neural Network (CNN) model with convolution layers, pooling layers, and fully connected layers.
9. Apply a ResNet (Residual Network) architecture using transfer learning to improve accuracy and reduce training time.
10. Train the model using the Training data, adjusting its internal parameters (weights) to minimize prediction error.
11. Validate and test the model using the Testing data to check how accurately it predicts tumor type.
12. Evaluate the model using metrics such as accuracy, precision, recall, and confusion matrix.
13. Use the trained model to predict the tumor class for a new, unseen MRI image.

Logic or Method Used :

The core logic is based on supervised image classification. Each MRI image is treated as an input, and its folder name acts as the correct label (ground truth). The system learns patterns directly from pixel data rather than manually written rules. During training, the model compares its predicted output with the actual label, calculates the error (loss), and adjusts its internal weights using backpropagation and an optimizer (such as Adam)

to gradually reduce this error. This process is repeated over multiple epochs until the model becomes accurate at recognizing visual patterns associated with each tumor type.

Model or Technique Applied :

Two deep learning techniques are applied in this project:

Convolutional Neural Network (CNN): Used to automatically extract important visual features from MRI images, such as edges, textures, and tumor-like shapes, without manual feature engineering.

ResNet (Residual Network): A deeper and more advanced CNN architecture that uses skip (residual) connections, allowing the model to train more effectively on complex image data and achieve higher classification accuracy.

Supporting tools include OpenCV for image preprocessing and TensorFlow (with Keras) for building, training, and evaluating both models.

Process Flow of the System :

The overall process flow of the system is as follows:

MRI Image Input → Image Preprocessing (Resizing, Normalization using OpenCV) → Feature Extraction (CNN Layers) → Classification (ResNet Model) → Model Training and Validation (TensorFlow) → Performance Evaluation (Accuracy, Precision, Recall) → Final Prediction Output (Tumor Type or No Tumor).

Source of Data :

```
# Source of Data
print("Dataset Name: Brain Tumor MRI Dataset")
print("Source: Kaggle - Brain Tumor MRI Dataset (Figshare, SARTAJ, Br35H)")
print("Format: Image dataset (.jpg), organized in class-labeled folders")
```

Number of Records/Samples :

```
#Number of Records/Samples
import os

TRAIN_DIR = "archive/Training"
TEST_DIR = "archive/Testing"

classes = sorted(os.listdir(TRAIN_DIR))
total_train = sum(len(os.listdir(os.path.join(TRAIN_DIR, c))) for c in classes)
total_test = sum(len(os.listdir(os.path.join(TEST_DIR, c))) for c in classes)

print("Total Training Samples:", total_train)
print("Total Testing Samples:", total_test)
print("Total Samples:", total_train + total_test)

for c in classes:
    print(c, "-> Train:", len(os.listdir(os.path.join(TRAIN_DIR, c))), "Test:", len(os.listdir(os.path.join(TEST_DIR, c)))
```

Features/Attributes :

```
#Features/Attributes
import os
import cv2

TRAIN_DIR = "archive/Training"
classes = sorted(os.listdir(TRAIN_DIR))

sample_img = os.path.join(TRAIN_DIR, classes[0], os.listdir(os.path.join(TRAIN_DIR, classes[0]))[0])
img = cv2.imread(sample_img)

print("Image Width:", img.shape[1])
print("Image Height:", img.shape[0])
print("Channels:", img.shape[2])
print("Classes (Labels):", classes)
```

Training and Testing Split :

```
#Training and Testing Split
import os

TRAIN_DIR = "archive/Training"
TEST_DIR = "archive/Testing"

classes = sorted(os.listdir(TRAIN_DIR))
total_train = sum(len(os.listdir(os.path.join(TRAIN_DIR, c))) for c in classes)
total_test = sum(len(os.listdir(os.path.join(TEST_DIR, c))) for c in classes)
total = total_train + total_test

print("Training Samples:", total_train, "(", round(total_train/total*100, 2), "%)")
print("Testing Samples:", total_test, "(", round(total_test/total*100, 2), "%)")
```

Data Format/Type :

```
#Data Format/Type
print("Data Type: Unstructured Image Data")
print("File Format: .jpg")
print("Label Type: Folder-based class labels")
```

RESULT :

```
PS C:\Users\kanno\Desktop\Brain_Tumor> python Brain.py
Dataset Name: Brain Tumor MRI Dataset
Source: Kaggle - Brain Tumor MRI Dataset (Figshare, SARTAJ, Br35H)
Format: Image dataset (.jpg), organized in class-labeled folders
Total Training Samples: 5600
Total Testing Samples: 1600
Total Samples: 7200
glioma -> Train: 1400 Test: 400
meningioma -> Train: 1400 Test: 400
notumor -> Train: 1400 Test: 400
pituitary -> Train: 1400 Test: 400
Image Width: 512
Image Height: 512
Channels: 3
Classes (Labels): ['glioma', 'meningioma', 'notumor', 'pituitary']
Training Samples: 5600 ( 77.78 % )
Testing Samples: 1600 ( 22.22 % )
Data Type: Unstructured Image Data
File Format: .jpg
Label Type: Folder-based class labels
```